

04_data_spark_tesla

June 8, 2026

1 Spark Exercise

Apache Spark is an excellent tool for data engineering projects due to its robust ability to process large-scale data efficiently through distributed computing. Spark's in-memory processing capabilities significantly enhance the speed of data operations, making it ideal for handling big data workloads. It supports various data sources and formats, offering versatility in data ingestion and transformation. Additionally, Spark's rich API supports multiple programming languages such as Python, Java, and Scala, catering to diverse developer preferences. Its ecosystem, which includes libraries for SQL, machine learning, and graph processing, provides a comprehensive suite for building complex data pipelines and analytics, making it a powerful and flexible choice for data engineering tasks.

Use Python, `pyspark` and `pandas` to explore Apache Spark RDD and DataFrame:

2 Spark RDD

Spark RDD (Resilient Distributed Dataset) is a fundamental data structure in Apache Spark that enables fault-tolerant, distributed processing of large datasets across multiple nodes in a cluster. Spark RDDs provide a higher-level abstraction for performing distributed data processing tasks, including both map (transformations) and reduce (aggregations) operations.

2.1 Import Necessary Libraries

```
[104]: from pyspark.sql import SparkSession
      from pyspark import SparkContext
      from pyspark.sql.functions import avg, substring, col
```

2.2 Spark Context and Session

Initialize Spark Context and Spark Session

```
[105]: spark = SparkSession.builder \
      .appName("Spark RDD Exercise") \
      .getOrCreate()

      sc = spark.sparkContext
```

2.3 Load Data into RDD

```
[106]: # CSV-Datei als Text-RDD laden
path = "../data/processed/tesla_stock_data_cleaned.csv"

rdd = sc.textFile(path)
```

2.4 Map Operation

Split data into individual parts and create key-value pairs

```
[107]: # Jede Zeile in ein Key-Value-Paar (Jahr, Volumen) umwandeln
def parse(line):
    cols = line.split(",")

    # hier wird der Header übersprungen
    if cols[0] == "Date":
        return None

    try:
        year = cols[0][:4]
        volume = float(cols[5])
        return (year, volume)
    except:
        return None

kv_rdd = rdd.map(parse).filter(lambda x: x is not None)
```

2.5 Reduce Operation

Reduce your key-value pairs

```
[108]: # Handelsvolumen pro Jahr aufsummieren
reduced_rdd = kv_rdd.reduceByKey(lambda a, b: a + b)
```

2.6 Collect Results

Because of lazy evaluation, the map-reduce operation is performed only now. Show what you calculated.

```
[109]: results = reduced_rdd.collect()

print("RDD Ergebnisse: Gesamtvolumen pro Jahr:")
if not results:
    print("Keine Daten gefunden!")
else:
    for year, total_volume in sorted(results):
```

```
print(f"{year} {total_volume:.2f}")
```

RDD Ergebnisse: Gesamtvolumen pro Jahr:

```
2020 57058095990.00
2021 20713137786.00
2022 21826184425.00
2023 34343805272.00
2024 23903815602.00
2025 24433675134.00
2026 5126041004.00
```

2.7 Save Results

```
[92]: output_path_rdd = "../data/results/rdd_output_v1"
      sc.parallelize(results).saveAsTextFile(output_path_rdd)
      print(f"\nRDD-Ergebnisse gespeichert unter: {output_path_rdd}")
```

RDD-Ergebnisse gespeichert unter: ../data/results/rdd_output_v1

3 Spark DataFrame

Spark DataFrame is a distributed collection of data organized into named columns, designed for efficient data manipulation and analysis in Apache Spark. It is used for various data processing tasks such as data ingestion, transformation, querying, and analysis in Apache Spark, providing a high-level abstraction that simplifies working with structured data.

3.1 Load Data into DataFrame

```
[110]: # CSV-Datei als Spark DataFrame laden
      df = spark.read.csv(
          "../data/processed/tesla_stock_data_cleaned.csv",
          header=True,
          inferSchema=True
      )
```

3.2 View DataFrame Schema

```
[111]: print("DataFrame Schema:")
      df.printSchema()
```

DataFrame Schema:

```
root
|-- Date: date (nullable = true)
|-- Open: double (nullable = true)
|-- High: double (nullable = true)
|-- Low: double (nullable = true)
```

```

|-- Close: double (nullable = true)
|-- Volume: integer (nullable = true)
|-- Daily_Return: double (nullable = true)
|-- Price_Range: double (nullable = true)
|-- MA_30: double (nullable = true)
|-- MA_100: double (nullable = true)

```

3.3 View DataFrame Data

```

[112]: print("Ersten 5 Zeilen:")
       df.show(5)

```

Ersten 5 Zeilen:

```

+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
|      Date|   Open|   High|   Low|  Close|   Volume|           Daily_Return|
Price_Range|MA_30|MA_100|
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
|2020-01-03|29.3667|30.2667| 29.128| 29.534|266920455|           NULL|
1.1387| NULL|  NULL|
|2020-01-06|29.3647| 30.104|29.3333|30.1027|152362485|
0.01925577300738124|0.77069999999999979| NULL|  NULL|
|2020-01-07| 30.76| 31.442|30.2237|31.2707|273137070|
0.03880050626687992|1.2182999999999993| NULL|  NULL|
|2020-01-08| 31.58|33.2333|31.2153|32.8093|467990895|
0.04920260819233335|2.0180000000000007| NULL|  NULL|
|2020-01-09|
33.14|33.2533|31.5247|32.0893|426947790|-0.02194499730259...|1.72860000000000037|
NULL|  NULL|
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
only showing top 5 rows

```

3.4 Filter Data

Performe a filter operation on a column

```

[113]: print("Tage mit sehr hohem handelsvolumen (>200 Mio.):")
       df_filtered = df.filter(df["Volume"] > 200000000)
       df_filtered.show(5)

```

Tage mit sehr hohem handelsvolumen (>200 Mio.):

```

+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
|      Date|   Open|   High|   Low|  Close|   Volume|           Daily_Return|

```

```

Price_Range|MA_30|MA_100|
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+
|2020-01-03|29.3667|30.2667| 29.128| 29.534|266920455|          NULL|
1.1387| NULL|  NULL|
|2020-01-07| 30.76| 31.442|30.2237|31.2707|273137070|
0.03880050626687992|1.2182999999999993| NULL|  NULL|
|2020-01-08| 31.58|33.2333|31.2153|32.8093|467990895|
0.04920260819233335|2.0180000000000007| NULL|  NULL|
|2020-01-09|
33.14|33.2533|31.5247|32.0893|426947790|-0.02194499730259...|1.72860000000000037|
NULL|  NULL|
|2020-01-13| 32.9|35.0433| 32.8| 34.99|399518205| 0.09766694795885411|
2.2433000000000005| NULL|  NULL|
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+
only showing top 5 rows

```

3.5 Group By and Aggregate

Performe a group by and aggregat operation

```

[114]: # Jahr aus dem Datum extrahieren
df_year = df.withColumn("Year", substring(col("Date"), 1, 4))

# Durchschnittlicher Schlusskurs und Volumen pro Jahr
df_grouped = df_year.groupBy("Year").agg(
    avg("Close").alias("avg_close"),
    avg("Volume").alias("avg_volume")
)

print("Aggregierte Ergebnisse pro Jahr:")
df_grouped.orderBy("Year").show()

```

```

Aggregierte Ergebnisse pro Jahr:
+---+-----+-----+
|Year|      avg_close|      avg_volume|
+---+-----+-----+
|2020| 96.93547341269844|2.2642101583333334E8|
|2021|259.99816269841267| 8.219499121428572E7|
|2022| 263.0930677290837| 8.695691005976096E7|
|2023| 217.4752400000001|      1.37375221088E8|
|2024|230.61480198412687| 9.485641111904761E7|
|2025| 356.9702399999999|      9.7734700536E7|
|2026| 401.8524234567899| 6.328445683950617E7|
+---+-----+-----+

```

3.6 Save DataFrame to Parquet

```
[115]: df_grouped.write.mode("overwrite").parquet("../data/results/stock_summary.  
        ↳parquet")
```